INFORME TECNICO

# Lemac DataLab

## Arquitectura, flujo de datos y revision frente a los requisitos MVP

Proyecto Android **limpio_moviles** · Kotlin + Jetpack Compose + Room

| | |
|---|---|
| **Preparado para** | Simon |
| **Fecha** | 7 de noviembre de 2026 |
| **Proyecto revisado** | /home/simon/AndroidStudioProjects/limpio_moviles |
| **Documentos revisados** | trabajo moviles.docx · Respuestas _MVP (2).docx · cartadeanimo.cl |
| **Alcance** | (1) Revisar si la app cumple lo pedido. (2) Explicar el flujo model → repository → datos → ViewModel → UI para estudiarlo. (3) Recomendar como crear un proyecto propio con la misma idea. |

**Como leer este informe.** Las secciones 1 a 6 responden la revision del trabajo (que tienes y que te falta). Las secciones 7 a 11 son la guia de estudio del flujo, en el orden que pediste: primero **model**, despues **repository**, despues las fuentes de datos, el ViewModel y la UI. La seccion 12 es la receta para crear tu propio proyecto.

# Contenido


1. Resumen ejecutivo y veredicto
2. Que dicen realmente los dos documentos adjuntos
3. Que es la app actual (Lemac DataLab / Carta de Animo)
4. Veredicto: tienes o no tienes lo que necesita el trabajo
5. Matriz de brechas requisito por requisito
6. Lo que si puedes reutilizar
7. Arquitectura por capas: el mapa completo
8. El flujo en orden: model, repository y demas
9. Un registro paso a paso, de la pantalla a la base de datos
10. Otros flujos: sincronizar, exportar CSV y recordatorio
11. Pruebas, calidad y limitaciones
12. Como crear tu propio proyecto con esta misma idea
13. Glosario y anexo de archivos y comandos


## Resumen en una pagina

| Pregunta | Respuesta corta |
|---|---|
| Que es la app que tienes? | Una app Android de autorregistro de bienestar (neurodesarrollo, DBT, animo y adicciones) llamada Lemac DataLab, con datos locales, sin login y con exportacion CSV. |
| Que pide *Respuestas_MVP*? | Un sistema de gestion documental de RR.HH. para una corporacion/area de salud: usuarios, 5 perfiles, matriz de permisos, documentos, aprobaciones, versionado, auditoria y el flujo RF12. |
| La app cumple eso? | **No.** No tiene login, roles, permisos, documentos, aprobaciones, historial ni auditoria. Es otro producto. |
| Y la web Cartadeanimo.cl? | Esa web si coincide con tu app (registro de animo para pacientes y profesionales). Hay una discrepancia entre los dos adjuntos que debes aclarar. |
| Que hago entonces? | Reutiliza la arquitectura (MVVM + Repository + Room) y construye el dominio documental desde cero. La seccion 12 trae la receta. |

# 1. Resumen ejecutivo y veredicto

Revisé el proyecto Android que has trabajado en los últimos días (`limpio_moviles`, nombre visible **Lemac DataLab**), los dos documentos que descargaste (`trabajo moviles.docx` y `Respuestas _MVP (2).docx`) y la página del cliente (`cartadeanimo.cl`). El objetivo era responder una pregunta concreta: **¿lo que tengo en Android Studio sirve para el trabajo que estamos viendo?**

> **Veredicto corto: no.** La app actual **no implementa** los requisitos de `Respuestas_MVP`. Es una app de autorregistro de bienestar, no un sistema de gestión documental de RR.HH. Lo que sí tienes es una **base técnica sólida y reutilizable** (arquitectura por capas, Room, repositorios, ViewModel, Compose, pruebas unitarias), pero el dominio, los roles, los permisos, los documentos y los flujos de aprobación hay que construirlos desde cero.

La confusión tiene una explicación: los dos adjuntos no describen el mismo producto.

- `trabajo moviles.docx` es una instrucción general: revisar la información nueva del cliente **Cartadeanimo.cl**, compararla con la app, corregir y recién después grabar el video. La web **sí** coincide con tu app: es una herramienta de registro de ánimo para pacientes y profesionales, creada por Terapéuticamente y desarrollada por Lemac Datalab.
- `Respuestas _MVP (2).docx` es otra cosa: habla de funcionarios con contrato, jefaturas, licencias médicas, expedientes, RR.HH. y del requisito **RF12** de modificación con trazabilidad. Eso es un **sistema documental interno**, no la app pública de ánimo.

Por eso, antes de grabar el video, conviene aclarar con el equipo o el docente **a cuál de los dos productos corresponde la entrega**. Si la entrega es la de `Respuestas_MVP`, la app actual no sirve tal como está. Si la entrega es Carta de Ánimo, vas bien encaminado, pero todavía te faltan cuentas reales, roles paciente/profesional, backend y la vista de “carta” que muestra la web.

## Qué sí tienes (y es valioso)

| Pieza | Por qué sirve para el próximo proyecto |
|---|---|
| Arquitectura MVVM + Repository | Separa UI, lógica de presentación y datos. Es exactamente el esqueleto que necesita un sistema documental. |
| Room (Entity, DAO, Database) | Ya sabes persistir y observar datos con `Flow`. Cambias la entidad y reutilizas el patrón. |
| Inyección de dependencias manual | `LemacApplication` actúa como contenedor. Para un MVP alcanza y se entiende mejor que Hilt. |
| ViewModel + StateFlow | Manejo de estado y corrutinas probado. Se replica tal cual. |
| Compose + Navigation | Pantallas, rutas y navegación ya funcionando. |
| WorkManager y notificaciones | Base para recordatorios, vencimientos y avisos de aprobación/rechazo. |
| Exportador CSV puro y testeable | Se puede adaptar a reportes de cumplimiento documental. |
| 19 pruebas unitarias | Demuestran que sabes separar lógica testeable de Android. |

> **En una frase:** no tienes el *producto* que pide `Respuestas_MVP`, pero sí tienes el *andamio* para construirlo. El trabajo real no es renombrar pantallas: es modelar usuarios, roles, permisos, documentos, estados y auditoría.

# 2. Qué dicen realmente los dos documentos adjuntos

## 2.1. `trabajo moviles.docx`

Es una comunicación de **Bastián Lisperguer** que adjunta “nueva información relevante entregada por la contraparte”. Los puntos concretos son:

- Página web del cliente: **Cartadeanimo.cl**.
- Respuestas a las dudas levantadas por los equipos.
- Requerimientos y definiciones actualizadas del proyecto.
- Antes de grabar el video, cada grupo debe **revisar** esta información y **contrastarla** con el estado actual de su aplicación.
- Ajustar lo que corresponda: **flujos, permisos, roles, validaciones, estados, reglas de negocio y acceso a la información**.
- Secuencia recomendada: revisar → comparar → corregir/actualizar → probar → recién después grabar.

Fuente: `/home/simon/Descargas/trabajo moviles.docx` (13 líneas).

## 2.2. `Respuestas _MVP (2).docx`

Es el documento de respuestas del cliente. Se organiza en tres bloques:

### Bloque 1 — Usuarios, perfiles y control de acceso

- La creación y habilitación de usuarios la hace un **administrador de RR.HH.**
- Solo acceden **funcionarios con contrato a plazo fijo o indefinido** de la Corporación/Área de Salud.
- **Perfiles oficiales del MVP:** Funcionario, Jefatura, Administrativo, Recursos Humanos y Administrador del sistema.
- **Actor de apoyo:** TI/Ciberseguridad (acceso excepcional, autorizado y trazable).
- La matriz de acceso también menciona un **Supervisor institucional** (reportes e indicadores de su ámbito).
- Cada perfil tiene acciones principales definidas (consultar, cargar, validar, aprobar/rechazar, registrar observaciones, etc.).

### Bloque 2 — Documentos, validación y aprobación

- **Matriz de acceso documental por perfil:** qué puede consultar cada uno y qué restricciones tiene.
- **Alcance por categoría documental:** contratos y anexos, certificados laborales, solicitudes de permisos, licencias médicas, antecedentes personales sensibles, historial de modificaciones y reportes institucionales.
- **Reglas generales:** el acceso se determina por perfil + dependencia + tipo de documento + función autorizada. El administrativo no tiene acceso transversal por defecto. El administrador del sistema no obtiene acceso automático al contenido sensible. Los cambios de cargo o dependencia gatillan revisión de permisos.

## Bloque 3 — RF12: modificación de información y trazabilidad

- Flujo de modificación con aprobación según el tipo de información.
- **Cambios simples** (teléfono, correo): se aplican inmediatamente si el campo está habilitado, con trazabilidad.
- **Datos personales oficiales** (nombre, RUT): requieren validación documental por RR.HH.
- **Contratos, anexos y documentos laborales:** la nueva versión queda **pendiente de revisión**.
- **Documentos cargados por administrativo:** no reemplazan al oficial hasta su validación.
- **Flujo:** solicitud → evaluación (directo o con revisión) → aprobación (publica, actualiza y conserva versión anterior) o rechazo (mantiene versión vigente, registra motivo, notifica y permite reenviar).
- **Auditoría mínima:** usuario responsable, fecha y hora, datos afectados, estado de la solicitud, resultado de la revisión y observaciones. El historial no puede ser alterado por usuarios ordinarios.

> **Idea clave del RF12:** no basta con “editar”. Hay que distinguir entre **edición directa** y **edición con aprobación**, y mientras una modificación está pendiente, la información anterior sigue vigente. Eso exige estados, versiones y auditoría, que la app actual no tiene.

# 3. Qué es la app actual (Lemac DataLab / Carta de Ánimo)

**Lemac DataLab** es una aplicación Android de autorregistro. La persona elige uno de cuatro ámbitos de bienestar, completa un formulario breve, guarda el registro y luego puede verlo en un historial, consultar un resumen, activar un recordatorio diario y cambiar su alias o foto.

| Aspecto | Estado real en el código |
|---|---|
| Nombre visible / paquete | Lemac DataLab / `com.example.limpio_moviles` |
| Lenguaje y UI | Kotlin 2.0.21 + Jetpack Compose (Material 3) |
| Ámbitos | Neurodesarrollo, Habilidades DBT, Estado de ánimo, Adicciones |
| Persistencia | Room ( `lemac_datalab.db` ) + SharedPreferences ( `perfil_lemac` ) + archivo `perfil.png` |
| Backend | Ninguno real: `FakeLemacApi` simula 700 ms y devuelve éxito |
| Autenticación | No existe. Solo un alias o seudónimo |
| Roles y permisos | No existen. Un único usuario implícito |
| Documentos | No gestiona documentos ni archivos adjuntos |
| Notificaciones | Recordatorio diario genérico a las 20:00 con WorkManager |
| Exportación | CSV local vía SAF (selector de documentos del sistema) |
| Privacidad | Sin permiso `INTERNET` , sin permiso `CAMERA` , sin permisos de almacenamiento |
| Pruebas | 19 pruebas unitarias JUnit 4 (validador, resumen y CSV) |

## Relación con Carta de Ánimo

La web **cartadeanimo.cl** es una herramienta gratuita de **Terapéuticamente**, desarrollada por **Lemac Datalab**. Permite a pacientes registrar cómo se sienten cada día y a profesionales revisar la evolución y añadir períodos clínicos. El ámbito “Estado de ánimo” de tu app es el que más se parece a esa idea.

Sin embargo, la app actual **no es todavía** la Carta de Ánimo completa: no tiene cuentas, no distingue paciente de profesional, no tiene backend ni la línea de tiempo clínica que muestra la web. Es un MVP académico del concepto de autorregistro.

> **Conclusión de esta sección:** la app es un producto de **bienestar/ánimo**. El documento `Respuestas_MVP` describe un producto de **gestión documental de RR.HH.** Son dominios distintos. Esa es la raíz de la brecha.

# 4. Veredicto: tienes o no tienes lo que necesita el trabajo

## 4.1. Si el trabajo es `Respuestas_MVP` (gestión documental)

> **No, no lo tienes.** La app no cubre los requisitos funcionales del documento. No hay usuarios, autenticación, perfiles, permisos, documentos, estados de aprobación, versionado, auditoría ni reportes de cumplimiento. La cobertura real es prácticamente **0%** de los requisitos específicos.

Lo único que se acerca es la **infraestructura técnica**: persistencia local, capa de repositorio, ViewModel y pruebas. Eso te ahorra tiempo, pero no reemplaza el desarrollo del dominio documental.

## 4.2. Si el trabajo es Carta de Ánimo (bienestar)

> **Parcialmente.** El dominio y la idea central (registrar el ánimo, privacidad, datos locales) están alineados. Pero faltan piezas que la web sí muestra: cuentas reales, roles paciente/profesional, backend, períodos clínicos, revisión profesional y la visualización de "carta" en el tiempo.

## 4.3. Discrepancia que debes aclarar antes de grabar

El documento `trabajo moviles.docx` dice que la información nueva viene de **Cartadeanimo.cl**, pero el archivo `Respuestas _MVP (2).docx` responde dudas de un **sistema documental de RR.HH.** No parecen ser el mismo producto. Antes de invertir horas, confirma con tu equipo o docente:

1. ¿La entrega evaluada es la app de ánimo (Carta de Ánimo) o el sistema documental (Respuestas_MVP)?
2. Si son dos proyectos distintos, ¿cuál se graba primero y con qué alcance?
3. Si el documento de respuestas está mal adjuntado, pide la versión correcta.

### Qué haría yo en tu lugar

| # | Acción | Por qué |
|---|---|---|
| 1 | Aclarar con el docente/equipo cuál es el producto de la entrega | Evita construir lo incorrecto y perder tiempo antes del video |
| 2 | Si es Carta de Ánimo: completar cuentas, roles y backend | Es la brecha principal frente a la web del cliente |
| 3 | Si es gestión documental: crear un proyecto nuevo reutilizando la arquitectura | El dominio es tan distinto que forzarlo en la app actual sería más caro que empezar limpio |
| 4 | Documentar la decisión (ADR) y el alcance del MVP | Trazabilidad para la defensa y para no repetir el análisis |

# 5. Matriz de brechas requisito por requisito

La siguiente tabla compara cada requisito del documento con el código real. “Evidencia” indica qué existe hoy en la app; “Acción” es lo mínimo que habría que construir.

| Requisito (Respuestas_MVP) | ¿Cumple? | Evidencia en el código actual | Acción recomendada |
|---|---|---|---|
| Creación y habilitación de usuarios por RR.HH. | **No** | No existe entidad `Usuario`; solo un alias en `SharedPreferences`. | Crear `Usuario`, `Rol` y pantallas de administración. Definir quién puede crear/desactivar. |
| Acceso solo a funcionarios con contrato fijo/indefinido | **No** | No hay autenticación ni modelo de contrato. | Login + validación de estado contractual + baja automática al término. |
| Perfiles: Funcionario, Jefatura, Administrativo, RR.HH., Administrador | **No** | Un único usuario implícito; no hay roles. | Enum de roles + matriz de permisos + navegación condicionada por rol. |
| TI/Ciberseguridad y Supervisor institucional | **No** | No existen. | Roles de solo lectura/auditoría, con acceso excepcional y trazable. |
| Matriz de acceso documental por perfil | **No** | No hay documentos ni permisos. | Motor de políticas: perfil + dependencia + tipo de documento + función autorizada. |
| Jerarquía institucional (dependencia, centros, unidades) | **No** | No existe modelo organizacional. | Entidades `Dependencia`, `Centro`, `Unidad` y relación con usuarios. |
| Carga, registro, clasificación y actualización de documentos | **No** | Solo se guardan registros de bienestar (`RegistroEntity`). | Entidad `Documento` + almacenamiento de archivos + categorías. |
| Asociar documentos a funcionarios | **No** | No hay relación documento–funcionario. | Relación `Documento.usuarioId` y expediente por funcionario. |
| Validación y aprobación de documentos | **No** | No hay flujo de revisión. | Estados y transiciones + bandeja de aprobación por rol. |
| Estados: vigente, pendiente de revisión, aprobado, rechazado | **No** | Solo existe el booleano `sincronizado`. | Enum `EstadoDocumento` y reglas de transición. |
| Versionado e historial de modificaciones | **No** | No hay versiones ni historial. | Tabla `VersionDocumento` inmutable + historial consultable. |
| RF12: modificación con aprobación según tipo | **No** | Editar un registro lo sobrescribe; no hay aprobación. | Flujo solicitud → evaluación → aprobación/rechazo con reglas por tipo. |
| Auditoría: usuario, fecha/hora, datos afectados, resultado, observaciones | **No** | No hay log de auditoría. | Tabla `Auditoria` append-only, consultable por perfiles autorizados. |
| Rechazo: mantener versión anterior, motivo, notificar, permitir reenvío | **No** | No existe el concepto de rechazo. | Estado rechazado + motivo + notificación + nueva solicitud. |
| Vencimientos y reportes de | **No** | No hay fechas de vencimiento ni reportes. | Campos de vigencia + WorkManager + reportes/CSV. |

| | | | |
|---|---|---|---|
| cumplimiento documental | | | |
| Notificaciones de aprobación, rechazo u observaciones | **Parcial** | Solo recordatorio diario genérico (`RecordatorioWorker`). | Notificaciones por evento (cambio de estado, nueva solicitud). |
| Persistencia local | **Sí (técnica)** | Room: `RegistroEntity`, `RegistroDao`, `LemacDatabase`. | Reutilizar el patrón con nuevas entidades. |
| Exportación de datos | **Sí (técnica)** | `RegistroCsvExporter` con BOM, CRLF y RFC 4180. | Adaptar a reportes de cumplimiento. |
| Backend / API real | **No** | `FakeLemacApi` solo simula 700 ms. | API REST/GraphQL + autenticación + sincronización real. |
| Seguridad y cifrado | **No** | Room sin cifrar; sin login; `allowBackup="true"`. | Cifrado (SQLCipher o servidor), RBAC, consentimiento y auditoría. |
| Pruebas de datos, repositorios y UI | **Parcial** | 19 pruebas unitarias de lógica pura; sin pruebas de Room, repos, ViewModel ni instrumentadas. | Agregar pruebas de DAO, repositorio, ViewModel y UI. |

> **Resumen de cobertura:** de los 21 puntos evaluados, **0** están implementados como requisito funcional del sistema documental, **2** existen solo como capacidad técnica reutilizable (Room y CSV), **2** son parciales (notificaciones y pruebas) y el resto está pendiente.

# 6. Lo que sí puedes reutilizar

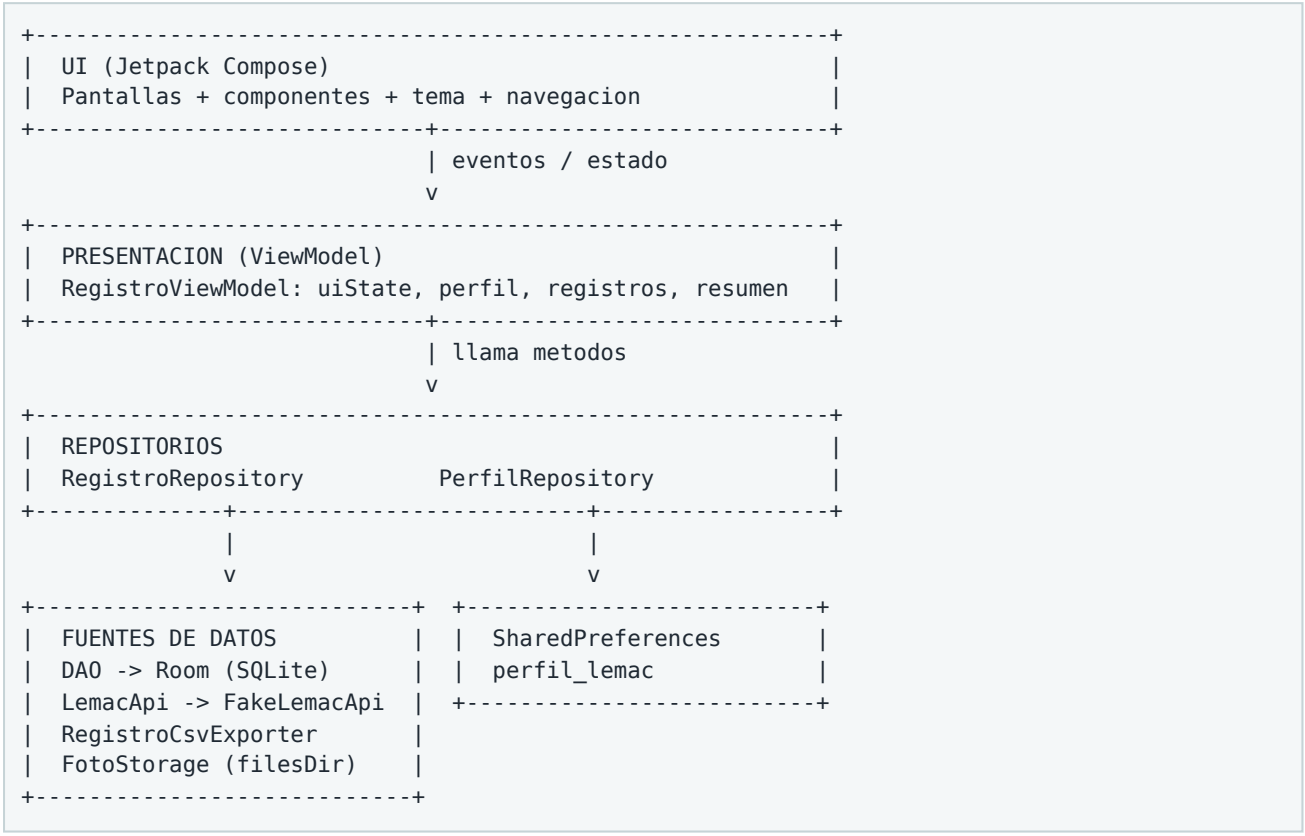
No todo es malo: la app está bien construida para un MVP y varias decisiones se trasladan casi sin cambios a un proyecto documental o a la Carta de Ánimo completa.

| Patrón | Dónde está | Cómo se reutiliza |
|---|---|---|
| Separación por capas | `model`, `data`, `viewmodel`, `ui` | Misma estructura con nuevas entidades y pantallas. |
| Repository | `RegistroRepository`, `PerfilRepository` | Crear `UsuarioRepository`, `DocumentoRepository`, `AuditoriaRepository`. |
| Room | `RegistroEntity`, `RegistroDao`, `LemacDatabase` | Nuevas tablas/migraciones; mismo patrón de `Flow` + `suspend`. |
| DI manual | `LemacApplication` | Contenedor de dependencias sin frameworks. |
| Estado de presentación | `RegistroViewModel` + `StateFlow` | Un ViewModel por flujo (documentos, aprobaciones, usuarios). |
| Navegación | `AppNavigation.kt` | Rutas por rol y pantallas protegidas. |
| WorkManager | `RecordatorioScheduler`, `RecordatorioWorker` | Vencimientos y recordatorios de revisión. |
| Exportador puro | `RegistroCsvExporter` | Reportes de cumplimiento en CSV. |
| Archivos y FileProvider | `FotoStorage` | Adjuntar documentos y compartir archivos de forma segura. |
| Pruebas unitarias | `app/src/test/...` | Probar validadores y cálculos sin emulador. |

> **Regla de oro:** reutiliza la **arquitectura**, no el **dominio**. Cambiar `RegistroEntity` por `DocumentoEntity` no es suficiente: hay que rediseñar reglas, estados, permisos y flujos.

# 7. Arquitectura por capas: el mapa completo

La app sigue una separación inspirada en **MVVM con repositorio**. La dependencia va en un solo sentido: la UI conoce al ViewModel, el ViewModel conoce a los repositorios y los repositorios conocen las fuentes de datos. Ninguna capa “salta” a otra.




## 7.1. Responsabilidad de cada capa

| Capa | Archivos | Qué hace | Qué NO debe hacer |
|---|---|---|---|
| Interfaz (UI) | `ui/screens/*`, `ui/components/*`, `ui/theme/*` | Mostrar estado, capturar eventos y navegar. | Acceder a Room o a SharedPreferences. |
| Presentación | `viewmodel/RegistroViewModel.kt` | Mantener estado, validar, lanzar corrutinas y coordinar repositorios. | Escribir SQL ni construir pantallas. |
| Repositorios | `data/repository/*` | Ofrecer operaciones de negocio y ocultar la fuente de datos. | Manejar estado de Compose. |
| Datos | `data/local/*`, `data/remote/*`, `util/*` | Persistir, consultar, simular red, generar CSV y guardar archivos. | Decidir textos o navegación. |

## 7.2. Inyección de dependencias manual

No se usa Hilt, Koin ni Dagger. `LemacApplication` actúa como contenedor:

```
class LemacApplication : Application() {
    val database by lazy { LemacDatabase.getDatabase(this) }
    val registroRepository by lazy {
        RegistroRepository(database.registroDao(), FakeLemacApi())
    }
    val perfilRepository by lazy { PerfilRepository(this) }

    override fun onCreate() {
        super.onCreate()
        NotificacionHelper.crearCanal(this)
    }
}
```

El `RegistroViewModelFactory` recibe esas dependencias y construye el ViewModel. Para un MVP es suficiente, se entiende y se explica bien en una defensa.

# 8. El flujo en orden: model, repository y demás

Esta es la parte que querías estudiar. Vamos de abajo hacia arriba, en el orden en que conviene entenderlo: primero el **model** (el vocabulario), después el **repository** (la puerta a los datos), después las **fuentes de datos**, luego el **ViewModel** (el coordinador) y al final la **UI**.

## 8.1. Model — el corazón y el vocabulario

El paquete `model/` no conoce Android, ni Room, ni Compose. Son clases Kotlin puras. Por eso se pueden probar sin emulador. Aquí se define **qué es un registro** y **qué reglas debe cumplir**.

### `Ambito.kt` — los cuatro mundos

```
enum class Ambito(val etiqueta: String, val descripcion: String) {
    NEURODESARROLLO("Neurodesarrollo", "Rutinas, tareas y autorregulacion"),
    DBT("Habilidades DBT", "Mindfulness, tolerancia, regulacion e interpersonal"),
    ANIMO("Estado de animo", "Check-in de emocion e intensidad"),
    ADICCIONES("Adicciones", "Impulsos, desencadenantes y acciones protectoras");

    companion object {
        fun desdeNombre(nombre: String?): Ambito =
            entries.firstOrNull { it.name == nombre } ?: NEURODESARROLLO
    }
}
```

**Qué explica:** un enum con etiqueta y descripción. `desdeNombre` convierte el texto que viene de la navegación o de Room en un valor seguro; si no coincide, cae en un valor por defecto. Eso evita *crashs* por datos viejos.

### `Catalogos.kt` — las opciones dependen del ámbito

```
object Catalogos {
    fun tipos(ambito: Ambito): List<String> = when (ambito) {
        Ambito.NEURODESARROLLO -> listOf("Rutina", "Tarea", "Sobrecarga", "Autorregulacion")
        Ambito.DBT -> listOf("Mindfulness", "Tolerancia al malestar", ...)
        Ambito.ANIMO -> listOf("Feliz", "Tranquilo", "Neutro", "Ansioso", "Triste",
"Irritable")
        Ambito.ADICCIONES -> listOf("Craving", "Impulso", "Recaida", "Objetivo cumplido")
    }

    fun rangoIntensidad(ambito: Ambito): IntRange =
        if (ambito == Ambito.ADICCIONES) 0..10 else 1..5
}
```

**Qué explica:** el catálogo es una función del ámbito. No hay listas duplicadas en la UI: la fuente de verdad está en el modelo. La intensidad cambia de rango según el ámbito (0–10 en adicciones, 1–5 en el resto).

### `RegistroUiState.kt` — el estado del formulario

```
data class RegistroUiState(
    val ambito: Ambito = Ambito.NEURODESARROLLO,
    val tipo: String = "",
    val intensidad: Int = 3,
    val contexto: String = "",
    val estrategia: String = "",
    val nota: String = "",
    val estadoObjetivo: String = "",
    val editandoId: Int? = null,
    val errores: Map<String, String> = emptyMap(),
    val guardadoExitoso: Boolean = false,
    val cargando: Boolean = false,
    val mensaje: String? = null
)
```

**Qué explica:** un *data class* inmutable que representa todo lo que la pantalla necesita saber. `errores` es un mapa campo → mensaje. `editandoId` distingue crear de editar. No contiene lógica: solo datos.

### `RegistroValidator.kt` — las reglas, sin UI

```
object RegistroValidator {
    fun validar(uiState: RegistroUiState): Map<String, String> = buildMap {
        if (uiState.tipo.isBlank()) put("tipo", "Selecciona un tipo")

        val rango = Catalogos.rangoIntensidad(uiState.ambito)
        if (uiState.intensidad !in rango)
            put("intensidad", "La intensidad debe estar entre ${rango.first} y
${rango.last}")

        if (uiState.ambito == Ambito.ADICCIONES && uiState.contexto.isBlank())
            put("contexto", "Indica el desencadenante")

        if (uiState.ambito == Ambito.DBT && uiState.estrategia.isBlank())
            put("estrategia", "Indica la habilidad practicada")

        if (uiState.ambito == Ambito.NEURODESARROLLO && uiState.estrategia.isBlank())
            put("estrategia", "Indica la estrategia utilizada")

        if (uiState.nota.length > 200)
            put("nota", "La nota no puede superar 200 caracteres")
    }
}
```

**Qué explica:** una función pura que recibe estado y devuelve errores. No muestra diálogos ni conoce Android. Por eso tiene 9 pruebas unitarias y es fácil de razonar.

### `ResumenAmbito.kt` y `ResumenCalculator.kt` — cálculos puros

```
data class ResumenAmbito(
    val ambito: Ambito,
    val cantidad: Int,
    val promedioIntensidad: Double,
    val ultimaFecha: Long?
)

object ResumenCalculator {
    fun calcular(registros: List<RegistroEntity>): List<ResumenAmbito> =
        Ambito.entries.map { ambito ->
```

```
            val delAmbito = registros.filter { it.ambito == ambito.name }
            ResumenAmbito(
                ambito = ambito,
                cantidad = delAmbito.size,
                promedioIntensidad = if (delAmbito.isEmpty()) 0.0
                    else delAmbito.sumOf { it.intensidad }.toDouble() / delAmbito.size,
                ultimaFecha = delAmbito.maxOfOrNull { it.fechaHora }
            )
        }
}
```

**Qué explica:** el resumen se calcula en el modelo, no en la pantalla. La UI solo dibuja el resultado. Esto permite probar los promedios sin emulador.

### `RegistroEntity.kt` — el modelo persistido

```
@Entity(tableName = "registros")
data class RegistroEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val usuarioAlias: String,
    val ambito: String,
    val tipo: String,
    val intensidad: Int,
    val contexto: String = "",
    val estrategia: String = "",
    val nota: String = "",
    val estadoObjetivo: String = "",
    val fechaHora: Long,
    val sincronizado: Boolean = false
)
```

**Qué explica:** aunque vive en `data/local`, es el modelo que se guarda. Cada fila de la tabla `registros` es un autorregistro. `id = 0` significa "nuevo" (Room lo autoincrementa). `sincronizado` marca si ya se envió (en este MVP, al backend falso).

> **Idea para memorizar:** el modelo es el **vocabulario** de la app. Si no sabes qué es un "ámbito", un "registro" o un "estado", no puedes entender el resto. Por eso se empieza aquí.

## 8.2. Repository — la puerta de los datos

El repositorio es una clase que **esconde de dónde vienen los datos**. El ViewModel no sabe si los datos están en Room, en memoria o en una API; solo llama al repositorio. Eso permite cambiar la fuente sin tocar la UI.

### `RegistroRepository.kt`

```
class RegistroRepository(
    private val dao: RegistroDao,
    private val api: LemacApi = FakeLemacApi()
) {
    val registros: Flow<List<RegistroEntity>> = dao.observarTodos()

    fun observarPorAmbito(ambito: Ambito): Flow<List<RegistroEntity>> =
        dao.observarPorAmbito(ambito.name)

    suspend fun guardar(registro: RegistroEntity): Boolean =
```

```
        if (registro.id == 0) dao.insertar(registro) != -1L
        else dao.actualizar(registro) > 0

    suspend fun eliminar(registro: RegistroEntity) = dao.eliminar(registro)
    suspend fun obtener(id: Int) = dao.obtenerPorId(id)
    suspend fun obtenerTodos() = dao.obtenerTodos()
    suspend fun eliminarTodos() = dao.eliminarTodos()

    suspend fun sincronizar(): Int {
        val pendientes = dao.obtenerPendientes()
        if (pendientes.isEmpty()) return 0
        api.sincronizar(pendientes).getOrThrow()
        dao.marcarSincronizados(pendientes.map { it.id })
        return pendientes.size
    }

    suspend fun cargarDemo(alias: String): Boolean { /* inserta 4 registros si esta vacio */
}
}
```

**Qué explica, método por método:**

| Método | Qué hace y por qué |
|---|---|
| `registros` | Expone un `Flow` del DAO. Cada cambio en la base emite una nueva lista; la UI se actualiza sola. |
| `observarPorAmbito` | Filtra por ámbito sin que el ViewModel escriba SQL. |
| `guardar` | Decide insertar o actualizar según `id`. Devuelve `Boolean` para que el ViewModel sepa si funcionó. |
| `eliminar` | Borra una fila. |
| `obtener` / `obtenerTodos` | Lecturas puntuales para editar o exportar. |
| `sincronizar` | Toma pendientes, llama a la API y los marca como sincronizados. |
| `cargarDemo` | Inserta datos de ejemplo solo si la tabla está vacía (evita duplicados). |

**`PerfilRepository.kt`**

```
class PerfilRepository(context: Context) {
    private val preferencias = context.applicationContext
        .getSharedPreferences("perfil_lemac", Context.MODE_PRIVATE)

    fun cargarAlias(): String = preferencias.getString("alias", "") ?: ""
    fun guardarAlias(alias: String) { preferencias.edit().putString("alias", alias).apply() }

    fun cargarFotoPath(): String? = preferencias.getString("foto_path", null)
    fun guardarFotoPath(path: String?) { /* put o remove */ }

    fun cargarRecordatorio(): Boolean = preferencias.getBoolean("recordatorio_activo", false)
    fun guardarRecordatorio(activo: Boolean) { /* putBoolean */ }
}
```

**Qué explica:** el perfil no es una entidad Room porque son pocos datos simples. Se guarda en `SharedPreferences`. El repositorio envuelve esas claves para que el ViewModel no maneje strings sueltos.

> **Regla:** el repositorio es el único que conoce la fuente de datos. Si mañana cambias Room por una API, cambias el repositorio, no la UI.

## 8.3. Fuentes de datos — Room, SharedPreferences, API falsa, CSV y archivos

### Room: `RegistroDao.kt`

```
@Dao
interface RegistroDao {
    @Query("SELECT * FROM registros ORDER BY fechaHora DESC, id DESC")
    fun observarTodos(): Flow<List<RegistroEntity>>

    @Query("SELECT * FROM registros WHERE ambito = :ambito ORDER BY fechaHora DESC")
    fun observarPorAmbito(ambito: String): Flow<List<RegistroEntity>>

    @Query("SELECT * FROM registros WHERE id = :id")
    suspend fun obtenerPorId(id: Int): RegistroEntity?

    @Insert suspend fun insertar(registro: RegistroEntity): Long
    @Update suspend fun actualizar(registro: RegistroEntity): Int
    @Delete suspend fun eliminar(registro: RegistroEntity)

    @Query("UPDATE registros SET sincronizado = 1 WHERE id IN (:ids)")
    suspend fun marcarSincronizados(ids: List<Int>)

    @Transaction
    suspend fun insertarDemoSiVacio(registros: List<RegistroEntity>): Boolean {
        if (contar() > 0) return false
        registros.forEach { insertar(it) }
        return true
    }
}
```

**Qué explica:** el DAO mezcla dos estilos. Las consultas que la UI observa devuelven `Flow`; las operaciones puntuales son `suspend`. `@Transaction` garantiza que “contar y luego insertar” sea atómico.

### Room: `LemacDatabase.kt`

```
@Database(entities = [RegistroEntity::class], version = 1, exportSchema = false)
abstract class LemacDatabase : RoomDatabase() {
    abstract fun registroDao(): RegistroDao

    companion object {
        @Volatile private var INSTANCE: LemacDatabase? = null

        fun getDatabase(context: Context): LemacDatabase =
            INSTANCE ?: synchronized(this) {
                Room.databaseBuilder(context.applicationContext,
                    LemacDatabase::class.java, "lemac_datalab.db")
                    .fallbackToDestructiveMigration()
                    .build().also { INSTANCE = it }
            }
    }
}
```

**Qué explica:** el patrón singleton con `@Volatile` y `synchronized` evita crear dos bases. `fallbackToDestructiveMigration` borra y recrea si cambia el esquema: cómodo para un MVP, peligroso en producción.

### API falsa: `LemacApi.kt`

```
interface LemacApi {
    suspend fun sincronizar(registros: List<RegistroEntity>): Result<Unit>
}

class FakeLemacApi : LemacApi {
    override suspend fun sincronizar(registros: List<RegistroEntity>): Result<Unit> {
        delay(700)
        return Result.success(Unit)
    }
}
```

**Qué explica:** el contrato existe, pero no hay red. `delay(700)` simula latencia para que la UI muestre el estado de carga. El manifiesto no declara `INTERNET`, así que la app no puede subir datos aunque quisiera.

### CSV: `RegistroCsvExporter.kt`

```
object RegistroCsvExporter {
    private const val BOM = "\uFEFF"
    private const val SALTO_LINEA = "\r\n"
    private const val CABECERA = "id,alias,ambito,tipo,intensidad,contexto," +
        "estrategia,nota,objetivo,fecha_hora_iso,fecha_hora_millis,sincronizado"

    fun generar(registros: List<RegistroEntity>): String { /* fila por registro */ }
    fun nombreArchivo(millis: Long = System.currentTimeMillis()): String =
        "lemac_datalab_" + /* yyyyMMdd_HHmmss */ ".csv"
}
```

**Qué explica:** es una clase pura: recibe una lista y devuelve texto. No conoce Android ni el sistema de archivos. Por eso se prueba con JUnit. Usa BOM UTF-8, CRLF y escapado RFC 4180 para que Excel no rompa acentos ni comas.

### Archivos: `FotoStorage.kt`

Guarda la foto de perfil como `perfil.png` en `filesDir`. Usa un archivo temporal y luego lo renombra para evitar corrupción. Lee la orientación EXIF y reduce la imagen para no llenar la memoria.

## 8.4. ViewModel — el coordinador

El ViewModel es el intermediario entre la UI y los repositorios. Mantiene el estado, lanza corrutinas y decide qué mensaje mostrar. Sobrevive a los cambios de configuración (por ejemplo, rotar la pantalla).

### Estado

```
class RegistroViewModel(
    application: Application,
    private val registroRepository: RegistroRepository,
    private val perfilRepository: PerfilRepository
) : AndroidViewModel(application) {

    var uiState by mutableStateOf(RegistroUiState())
        private set
```

```
    var perfil by mutableStateOf(PerfilUiState())
        private set

    var fotoPerfil by mutableStateOf<Bitmap?>(null)
        private set

    val registros: StateFlow<List<RegistroEntity>> =
        registroRepository.registros.stateIn(
            scope = viewModelScope,
            started = SharingStarted.WhileSubscribed(5_000),
            initialValue = emptyList()
        )

    val resumen: StateFlow<List<ResumenAmbito>> =
        registroRepository.registros
            .map { ResumenCalculator.calcular(it) }
            .stateIn(viewModelScope, SharingStarted.WhileSubscribed(5_000),
                ResumenCalculator.calcular(emptyList()))
}
```

**Qué explica:** hay dos mecanismos de estado a propósito:

| Mecanismo | Para qué se usa |
|---|---|
| `mutableStateOf` | Estado del formulario y del perfil: cambios simples que la UI lee directamente con Compose. |
| `StateFlow` + `stateIn` | Listas que vienen de Room y deben reaccionar a cambios de la base. `WhileSubscribed(5000)` deja de observar si la UI no está visible. |

## Guardar: el método más importante

```
fun guardar() {
    val errores = RegistroValidator.validar(uiState)
    if (errores.isNotEmpty()) {
        uiState = uiState.copy(errores = errores)
        return
    }

    val formulario = uiState
    uiState = uiState.copy(cargando = true, guardadoExitoso = false, errores = emptyMap())

    viewModelScope.launch {
        val existente = formulario.editandoId?.let { id ->
            runCatching { registroRepository.obtener(id) }.getOrNull()
        }
        val registro = RegistroEntity(
            id = formulario.editandoId ?: 0,
            usuarioAlias = perfil.alias.ifBlank { "Usuario de prueba" },
            ambito = formulario.ambito.name,
            tipo = formulario.tipo.trim(),
            intensidad = formulario.intensidad,
            contexto = formulario.contexto.trim(),
            estrategia = formulario.estrategia.trim(),
            nota = formulario.nota.trim(),
            estadoObjetivo = formulario.estadoObjetivo.trim(),
            fechaHora = existente?.fechaHora ?: System.currentTimeMillis(),
            sincronizado = false
        )
        val guardado = registroRepository.guardar(registro)
        uiState = uiState.copy(
            cargando = false,
```

```
            guardadoExitoso = guardado,
            mensaje = if (guardado) "Registro guardado" else "No se pudo guardar"
        )
    }
}
```

**Qué explica:** valida primero, sin tocar la base. Si hay errores, no hace nada más. Si todo está bien, construye la entidad, llama al repositorio dentro de `viewModelScope` y actualiza el estado. La UI reacciona al nuevo estado.

## 8.5. Application, Activity y Factory

| Archivo | Responsabilidad |
|---|---|
| `LemacApplication.kt` | Contenedor de dependencias y creación del canal de notificaciones. |
| `MainActivity.kt` | Única Activity. Aplica el tema y monta `LemacApp()`. |
| `RegistroViewModelFactory` | Construye el ViewModel con la Application y los repositorios. |

## 8.6. UI y navegación — la punta del iceberg

`AppNavigation.kt` define las rutas y el `NavHost`. El destino inicial depende del perfil: si el alias está vacío, va a `onboarding`; si no, a `home`.

```
object Rutas {
    const val ONBOARDING = "onboarding"
    const val HOME = "home"
    const val HISTORIAL = "historial"
    const val RESUMEN = "resumen"
    const val AJUSTES = "ajustes"
    const val REGISTRO_PATRON = "registro/{ambito}?id={id}"

    fun registro(ambito: Ambito, id: Int? = null) =
        "registro/${ambito.name}?id=${id ?: -1}"
}
```

Las pantallas no acceden a Room. Reciben el ViewModel, leen el estado y le envían eventos. Los componentes reutilizables (tarjetas, botones, selectores) tampoco conocen la base de datos.

| Pantalla | Qué hace |
|---|---|
| `OnboardingScreen` | Alias, aviso de privacidad, foto opcional y botón para comenzar. |
| `HomeScreen` | Saludo, foto, accesos rápidos, tarjetas de ámbito y carga de datos demo. |
| `RegistroScreen` | Formulario: chips de tipo, intensidad, campos, validación y guardado. |
| `HistorialScreen` | Lista, filtro por ámbito, editar, eliminar con diálogo. |
| `ResumenScreen` | Totales, promedios y actividad por ámbito; botón de exportar CSV. |
| `AjustesScreen` | Alias, foto, recordatorio, exportación, sincronización, datos demo y borrado. |

# 9. Un registro paso a paso, de la pantalla a la base de datos

Este es el recorrido completo de “guardar un registro nuevo”. Sirve para explicar la arquitectura en una defensa sin memorizar código.

| # | Paso | Qué ocurre y en qué archivo |
|---|---|---|
| 1 | El usuario toca una tarjeta de ámbito | `HomeScreen` llama `onAmbito(ambito)`. `AppNavigation` navega a `Rutas.registro(ambito)`. |
| 2 | Se abre el formulario | `RegistroScreen` recibe `ambito` e `id = null`. En `LaunchedEffect` llama `viewModel.prepararFormulario(ambito, null)`. |
| 3 | El usuario completa | Cada campo llama `actualizarTipo`, `actualizarIntensidad`, etc. Todos actualizan `uiState` con `copy()`. |
| 4 | Toca “Guardar” | La pantalla llama `viewModel.guardar()`. |
| 5 | Validación | `RegistroValidator.validar(uiState)` devuelve un mapa de errores. Si hay errores, se muestran y el flujo termina. |
| 6 | Construcción de la entidad | El ViewModel arma `RegistroEntity` con el alias del perfil, el ámbito, los campos recortados, `fechaHora` y `sincronizado = false`. |
| 7 | Repositorio | `registroRepository.guardar(registro)` decide: si `id == 0`, `dao.insertar`; si no, `dao.actualizar`. |
| 8 | Room | El DAO ejecuta el SQL contra `lemac_datalab.db`. Room devuelve el id o la cantidad de filas afectadas. |
| 9 | Nuevo estado | El ViewModel actualiza `uiState` con `guardadoExitoso = true` y un mensaje. |
| 10 | La lista se entera sola | El `Flow` de `registros` emite la nueva lista. `resumen` se recalcula. La UI se recompone. |
| 11 | Confirmación | `LaunchedEffect(guardadoExitoso)` muestra el mensaje o vuelve atrás. |

> **Para explicar en 20 segundos:** “La pantalla no guarda nada. Le avisa al ViewModel. El ViewModel valida con el modelo, arma la entidad y le pide al repositorio que la guarde. El repositorio usa el DAO de Room. Cuando Room escribe, el `Flow` avisa y la lista se actualiza sola.”

## Editar un registro

El flujo es casi el mismo, con dos diferencias: la navegación incluye `?id=5`, `prepararFormulario(ambito, 5)` carga el registro y llena el estado; al guardar, `id = 5` hace que el repositorio actualice en vez de insertar. La `fechaHora` original se conserva y `sincronizado` vuelve a `false` para indicar que hay cambios pendientes.

# 10. Otros flujos: sincronizar, exportar CSV y recordatorio

## 10.1. Sincronizar con la API simulada

```
fun sincronizar() {
    viewModelScope.launch {
        uiState = uiState.copy(cargando = true)
        runCatching { registroRepository.sincronizar() }
            .onSuccess { cantidad ->
                uiState = uiState.copy(cargando = false,
                    mensaje = if (cantidad > 0) "Sincronizados $cantidad registros"
                              else "No hay registros pendientes")
            }
            .onFailure { uiState = uiState.copy(cargando = false,
                mensaje = "No se pudo sincronizar") }
    }
}
```

**Recorrido:** ViewModel → repositorio → `dao.obtenerPendientes()` → `api.sincronizar(pendientes)` (espera 700 ms) → `dao.marcarSincronizados(ids)`. No hay red real.

## 10.2. Exportar a CSV

1. El usuario pulsa “Exportar datos (CSV)” en Ajustes o Resumen.
2. `BotonExportarCsv` lanza `ActivityResultContracts.CreateDocument("text/csv")` (SAF).
3. El sistema abre el selector de documentos; el usuario elige dónde guardar.
4. `viewModel.exportarCsv(uri)` lee todos los registros con `registroRepository.obtenerTodos()`.
5. `RegistroCsvExporter.generar(lista)` produce el texto.
6. `contentResolver.openOutputStream(uri)` escribe el archivo en `Dispatchers.IO`.
7. Se muestra “Datos exportados (N registros)”.

No se pide permiso de almacenamiento porque SAF otorga acceso temporal al archivo elegido.

## 10.3. Recordatorio diario

1. El usuario activa el recordatorio en Ajustes.
2. `PerfilRepository.guardarRecordatorio(true)` lo persiste.
3. `RecordatorioScheduler.programar(context)` crea un `PeriodicWorkRequest` de 1 día con retraso inicial hasta las 20:00.
4. WorkManager ejecuta `RecordatorioWorker.doWork()`.
5. `NotificacionHelper.mostrar(context)` publica una notificación genérica si hay permiso `POST_NOTIFICATIONS`.

**Limitación:** la notificación no distingue eventos (aprobación, rechazo, vencimiento). Para el sistema documental habría que crear notificaciones por evento.

## 10.4. Foto de perfil

`SelectorFotoPerfil` usa `TakePicturePreview`, que no requiere permiso `CAMERA`. El bitmap se pasa a `viewModel.actualizarFoto(bitmap)`, que guarda en `Dispatchers.IO` con `FotoStorage.guardar` y persiste la ruta en SharedPreferences.

# 11. Pruebas, calidad y limitaciones

## 11.1. Pruebas existentes

| Archivo | Pruebas | Qué cubre |
|---|---|---|
| `RegistroValidatorTest` | 9 | Tipo obligatorio, rangos de intensidad, campos por ámbito y límite de 200 caracteres. |
| `ResumenCalculatorTest` | 4 | Lista vacía, promedios, última fecha y ámbito sin datos. |
| `RegistroCsvExporterTest` | 6 | BOM y cabecera, fila completa, escapado RFC 4180, lista vacía, nombre de archivo y CRLF. |
| **Total** | **19** | JUnit 4, ejecutables con `./gradlew :app:testDebugUnitTest`. |

## 11.2. Qué no está probado

- Room (DAO y base de datos).
- Repositorios.
- ViewModel y corrutinas.
- WorkManager y notificaciones.
- Interfaz (no hay pruebas instrumentadas ni `androidTest`).

## 11.3. Limitaciones reales del MVP

- No hay backend real ni autenticación.
- La base Room no está cifrada.
- `fallbackToDestructiveMigration` puede borrar datos al cambiar el esquema.
- El CSV es texto plano e incluye alias y notas.
- No es un instrumento de diagnóstico ni reemplaza atención profesional.
- No hay cuentas ni separación paciente/profesional (para Carta de Ánimo).
- No hay documentos, roles, permisos ni auditoría (para el sistema documental).

> **Para producción** harían falta: autenticación, autorización por roles, cifrado en reposo y en tránsito, consentimiento informado, auditoría, gestión de sesiones, y cumplimiento normativo de datos de salud. Nada de eso está en el MVP.

# 12. Cómo crear tu propio proyecto con esta misma idea

Tu objetivo es “crear uno mismo, pero diferente a mi manera”. La buena noticia es que la **forma** de organizar el código se repite. Lo que cambia es el **dominio**. Esta es la receta, en el mismo orden que estudiamos.

## 12.1. Receta general (7 pasos)

| # | Paso | Qué haces |
|---|---|---|
| 1 | Define el dominio | Escribe las entidades, enums y reglas en `model/`. Sin Android. Pregúntate: ¿qué sustantivos y estados existen? |
| 2 | Define los contratos | Interfaces de DAO, API y repositorios. Piensa en qué operaciones necesita la app, no en cómo se implementan. |
| 3 | Implementa las fuentes | Room (Entity/DAO/Database), API real o fake, archivos. Aquí sí usas Android. |
| 4 | Implementa el repositorio | Une las fuentes y aplica reglas de negocio. El ViewModel solo ve el repositorio. |
| 5 | Implementa el ViewModel | Estado inmutable + eventos + corrutinas. Un ViewModel por flujo grande. |
| 6 | Implementa la UI | Compose: pantallas que leen estado y emiten eventos. Componentes reutilizables. |
| 7 | Conecta y prueba | Application como contenedor, Factory, navegación, pruebas unitarias de la lógica pura. |

## 12.2. Ejemplo aplicado al sistema documental

Si decides construir el sistema de `Respuestas_MVP`, este sería un punto de partida razonable. No es código final: es el mapa de entidades y capas.

### Modelo propuesto

| Entidad / enum | Campos o valores clave |
|---|---|
| `Usuario` | id, rut, nombre, correo, dependenciaId, rol, estado (activo/inactivo), tipoContrato. |
| `Rol` | FUNCIONARIO, JEFATURA, ADMINISTRATIVO, RRHH, ADMIN_SISTEMA, TI_CIBERSEGURIDAD, SUPERVISOR. |
| `Permiso` | rol, tipoDocumento, accion (ver, cargar, validar, aprobar, rechazar, auditar), ámbito. |
| `Dependencia` / `Centro` / `Unidad` | Jerarquía institucional para limitar el acceso. |
| `Documento` | id, usuarioId, categoria, titulo, archivo, estado, versionActual, fechaVencimiento. |
| `VersionDocumento` | documentoId, numeroVersion, archivo, autorId, fecha, comentario. |
| `EstadoDocumento` | VIGENTE, PENDIENTE_REVISION, APROBADO, RECHAZADO, ARCHIVADO. |

| | |
|---|---|
| `SolicitudModificacion` | documentoId, solicitanteId, tipoCambio, datosPropuestos, estado, revisorId, motivoRechazo. |
| `Auditoria` | usuarioId, fechaHora, entidad, entidadId, accion, datosAntes, datosDespues, resultado, observaciones. |

### Reglas que no pueden faltar

- Un cambio simple se aplica directo; un dato oficial o documento validado pasa a `PENDIENTE_REVISION`.
- Mientras está pendiente, la versión vigente sigue siendo la anterior.
- Aprobar publica la nueva versión y conserva la anterior en el historial.
- Rechazar mantiene la versión anterior, registra motivo y notifica.
- Toda acción escribe en `Auditoria`; el historial es append-only.
- El acceso se calcula por rol + dependencia + tipo de documento + función.
- El administrador del sistema no ve contenido sensible por defecto.

### Estructura de paquetes sugerida

```
com.example.gestiondocumental/
├── model/
│   ├── Usuario.kt, Rol.kt, Permiso.kt
│   ├── Documento.kt, EstadoDocumento.kt, VersionDocumento.kt
│   ├── SolicitudModificacion.kt, Auditoria.kt
│   └── Validadores.kt
├── data/
│   ├── local/      (Entidades Room, DAOs, Database)
│   ├── remote/     (ApiService, DTOs, AuthInterceptor)
│   └── repository/ (UsuarioRepository, DocumentoRepository,
│                    PermisoRepository, AuditoriaRepository)
├── viewmodel/
│   ├── SesionViewModel.kt, DocumentosViewModel.kt
│   └── AprobacionesViewModel.kt
├── navigation/AppNavigation.kt
├── ui/
│   ├── screens/    (Login, Home por rol, Documentos, Detalle,
│   │                BandejaAprobacion, Auditoria, Usuarios)
│   └── components/
└── util/
```

## 12.3. Qué cambia frente a Lemac DataLab

| Lemac DataLab | Sistema documental |
|---|---|
| Un usuario implícito (alias) | Muchos usuarios con login y roles |
| Datos locales y fake API | Backend real + sincronización |
| Registro de bienestar | Documentos, versiones y solicitudes |
| Sin estados de negocio | Máquina de estados (vigente, pendiente, aprobado, rechazado) |
| Sin auditoría | Auditoría obligatoria append-only |
| Recordatorio genérico | Notificaciones por evento y vencimientos |
| Privacidad local | RBAC, cifrado y trazabilidad |

> **Consejo:** no empieces por las pantallas. Empieza por el **modelo** y por las **reglas de estado**. Si el modelo está bien, el resto (repositorio, ViewModel, UI) se ordena casi solo. Es exactamente lo que hace bien Lemac DataLab y lo que deberías replicar.

# 13. Glosario y anexo

## 13.1. Glosario mínimo

| Término | Qué significa |
|---|---|
| MVVM | Model–View–ViewModel: separa datos, interfaz y estado de presentación. |
| Repository | Clase que oculta la fuente de datos y ofrece operaciones de negocio. |
| Entity | Clase que representa una tabla de la base de datos. |
| DAO | Data Access Object: interfaz con las consultas a la base. |
| Flow | Flujo de datos de Kotlin que emite valores a lo largo del tiempo. |
| StateFlow | Flow con estado actual, ideal para la UI. |
| ViewModel | Componente que mantiene el estado y sobrevive a cambios de configuración. |
| Compose | Kit de UI declarativa de Android. |
| DI | Inyección de dependencias: pasar las piezas que una clase necesita. |
| Room | Capa de persistencia sobre SQLite. |
| WorkManager | Programador de tareas en segundo plano. |
| SAF | Storage Access Framework: selector de documentos del sistema. |
| FileProvider | Mecanismo seguro para compartir archivos entre apps. |
| KSP | Procesador de símbolos de Kotlin usado por Room. |
| RBAC | Control de acceso basado en roles. |
| ADR | Architecture Decision Record: registro de una decisión técnica. |

## 13.2. Archivos clave del proyecto

| Ruta | Para qué sirve |
|---|---|
| `app/src/main/java/.../model/` | Entidades, enums, estados, validadores y cálculos puros. |
| `app/src/main/java/.../data/local/` | Room: entidad, DAO, base y exportador CSV. |
| `app/src/main/java/.../data/remote/` | Contrato de API y simulación. |
| `app/src/main/java/.../data/repository/` | Repositorios de registros y perfil. |
| `app/src/main/java/.../viewmodel/` | Estado y lógica de presentación. |
| `app/src/main/java/.../navigation/` | Rutas y NavHost. |
| `app/src/main/java/.../ui/screens/` | Seis pantallas Compose. |
| `app/src/main/java/.../ui/components/` | Componentes reutilizables. |
| `app/src/main/java/.../util/` | Foto, notificaciones y recordatorios. |
| `app/src/test/java/...` | 19 pruebas unitarias. |
| `docs/GUIA_ESTUDIO.md` | Guía extendida de estudio (1267 líneas). |
| `docs/DISERTACION.md` | Guion de exposición y demo. |

## 13.3. Comandos útiles

```
# Compilar el APK de debug
./gradlew :app:assembleDebug

# Instalar en emulador o dispositivo
./gradlew :app:installDebug

# Ejecutar las 19 pruebas unitarias
./gradlew :app:testDebugUnitTest

# Ver el reporte HTML de pruebas
# app/build/reports/tests/testDebugUnitTest/index.html
```

## 13.4. Nota sobre el estado del repositorio

Al momento de la revisión, el repositorio local tiene dos archivos modificados sin confirmar: `ui/components/BotonExportarCsv.kt` (se quitó un comentario) y `util/NotificacionHelper.kt` (se agregó `@SuppressLint("MissingPermission")`). Son cambios menores y no afectan el análisis. El último commit es `f7120c5` (“Estabilización de arquitectura Compose, Room y ViewModels”).

> **Cierre.** La app actual no es el sistema documental que describe `Respuestas_MVP`. Pero la arquitectura que construiste es un buen punto de partida para tu próximo proyecto. Estudia el orden **model → repository → datos → ViewModel → UI**, y replica ese esqueleto con tu propio dominio.